

Advanced Statistics - Assignment 3

Daniele Ghezzi - 2195461
Physics of Data - University of Padua

June 2, 2026

You will analyze a simulated binary-response dataset using Bayesian Bernoulli regression. The data consist of observations

$$D = \{(y_i, x_i)\}_{i=1}^N$$

where $y_i \in \{0, 1\}$ is a binary outcome and x_i is a real-valued explanatory variable. In this assignment you will fit Bayesian logistic regression models using either JAGS or Stan. You are expected to run multiple chains, diagnose convergence and mixing, and justify the reliability of your posterior summaries.

Model

The main model is

$$y_i | \beta_0, \beta_1 \sim \text{Bernoulli}(\pi_i),$$

$$\text{logit}(\pi_i) = \beta_0 + \beta_1 x_i,$$

with independent priors

$$\beta_0 \sim \mathcal{N}(0, 3^2),$$

$$\beta_1 \sim \mathcal{N}(0, 3^2).$$

Equivalently,

$$\pi_i = \frac{\exp(\beta_0 + \beta_1 x_i)}{1 + \exp(\beta_0 + \beta_1 x_i)}.$$

Theoretical Analysis Bayesian Bernoulli regression is a probabilistic model used to fit and predict binary outcomes $y_i \in \{0, 1\}$. In this framework, the model parameters are treated as random variables. Their posterior joint distribution is obtained through Bayes theorem by updating prior knowledge with the observed data. In this model, the probability of a single observation y_i , given a success probability π_i , follows a Bernoulli distribution:

$$P(y_i | \pi_i) = \pi_i^{y_i} (1 - \pi_i)^{1-y_i}$$

Assuming the observations are independent, the overall likelihood function for the dataset D is the product of the individual probabilities:

$$p(D | \{\pi_i\}_{i=1}^N) = \prod_{i=1}^N \pi_i^{y_i} (1 - \pi_i)^{1-y_i} \quad (1)$$

Since the probabilities π_i must strictly fall between 0 and 1, the linear predictor (formed by the covariate x_i and the regression coefficients) is mapped to the probability space using a link function, which in this case is the logit function.

Before proceeding with the model implementation, we first evaluate the general descriptive statistics of the provided dataset with the following R script.

```
1 library(cmdstanr) # we use STAN
2 library(coda)
3 library(posterior)
4 library(ggplot2)
5 library(loo)
6
7 set.seed(42)
8 data <- read.csv("ex3/logistic_data.csv")
9 summary(data)
10 data <- list(
11   N = nrow(data),
12   x = data$x,
13   y = data$y
14 )
15 sprintf("Number of observations: %d", data$N)
16
17 df <- as.data.frame(data)
18 p2 <- ggplot(df, aes(x = x, y = y)) +
19   geom_point(alpha = 0.5, size = 2) +
20   labs(title = "Data Plot",
21        x = "Covariate (x)", y = "Outcome (y)") +
22   theme_minimal()
23 print(p2)
```

```
      y      x
Min.  :0.00  Min.  :-2.35841
1st Qu.:0.00 1st Qu.: -0.53137
Median :0.00 Median : 0.02411
Mean   :0.45 Mean   : 0.08113
3rd Qu.:1.00 3rd Qu.: 0.85017
Max.   :1.00 Max.   : 2.44390
[1] "Number of observations: 120"
```

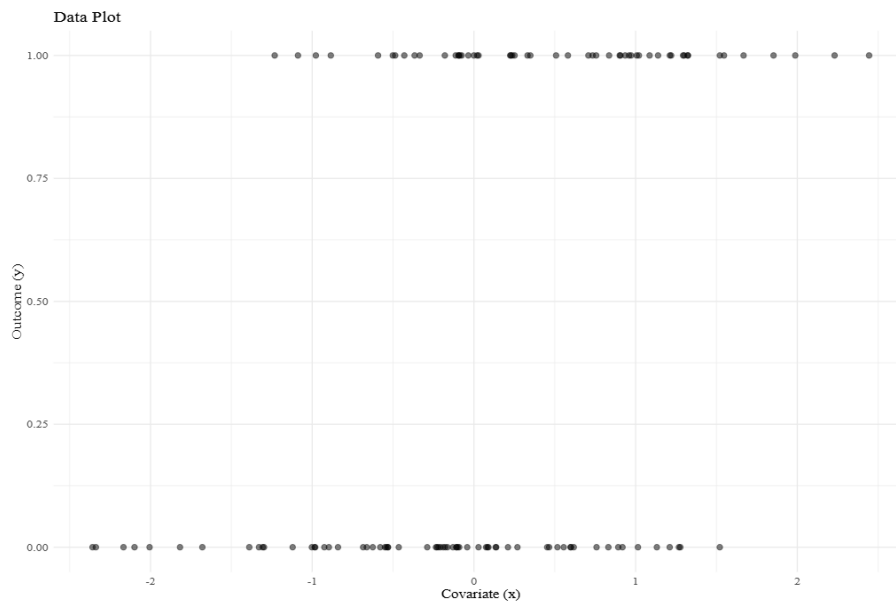


Figure 1: Data plot illustrating the relationship between the continuous covariate x and the binary outcome y . The distribution shows a clear tendency for positive covariate values to be associated with the positive outcome class ($y_i = 1$).

1 Posterior Formulation and Implementation

Posterior formulation and implementation. Then implement the logistic Bernoulli regression model in JAGS or Stan. Include the model code in your submission and clearly state which software and interface you used.

In this assignment, we use **Stan** (via the `cmdstanr` package in R) to perform the Bayesian logistic regression. Stan is well-suited for this task because the parameter space is continuous, allowing for highly efficient posterior sampling using Hamiltonian Monte Carlo (HMC).

We define the model's code with N observations, the real variable x and the binary outcome $y \in \{0, 1\}$. Next, we declare the parameters β_0 and β_1 and assign them their respective independent normal priors. The likelihood is defined using the `bernoulli_logit` function, which is

$$P(y_i | x_i, \beta_0, \beta_1) = \frac{\exp(y_i(\beta_0 + \beta_1 x_i))}{1 + \exp(\beta_0 + \beta_1 x_i)} \quad (2)$$

Finally, the `generated quantities` block evaluates two additional variables at each iteration: `y_rep`, which contains the sample posterior predictive simulations, and `log_lik`, which stores the point-wise log-likelihoods required for model comparison.

Here is the model code, saved in the file `logistic_regressionM1.stan`:

```
1 data {
2   int<lower=1> N;
3   vector[N] x;
4   array[N] int<lower=0, upper=1> y;
5 }
6 parameters {
7   real beta0;
8   real beta1;
9 }
10 model {
11   beta0 ~ normal(0, 3);
12   beta1 ~ normal(0, 3);
13   y ~ bernoulli_logit(beta0 + beta1 * x);
14 }
15 generated quantities {
16   array[N] int y_rep;
17   vector[N] log_lik;
18   for (i in 1:N) {
19     y_rep[i] = bernoulli_logit_rng(beta0 + beta1 * x[i]); // samples
20     log_lik[i] = bernoulli_logit_lpmf(y[i] | beta0 + beta1 * x[i]); // log pmf
21   }
22 }
```

In the R script, we then compile the Stan file:

```
1 # 1. Posterior formulation and implementation
2 mod1 <- cmdstan_model("ex3/logistic_regressionM1.stan")
```

Compiling Stan program...

2 MCMC Fitting and Diagnostics

MCMC fitting and diagnostics. Run at least four MCMC chains from dispersed initial values. Use a sufficiently long warm-up/burn-in period and sampling period. Report and discuss the main convergence and mixing diagnostics, including trace plots, $\hat{R}$, effective sample size, and autocorrelation. If using Stan, also report whether there were divergent transitions and comment on any relevant sampler diagnostics.

We ran four parallel MCMC chains, each with a warm-up period of 1000 iterations followed by

10000 sampling iterations. The chains converged and mixed well. Furthermore, Stan's HMC diagnostics detected zero divergent transitions, confirming that the sampling trajectories have no errors. The diagnostics evaluate to the following:

- **Trace Plots:** The trace plots exhibit stationary behavior across all chains, with dense and well mixed paths.

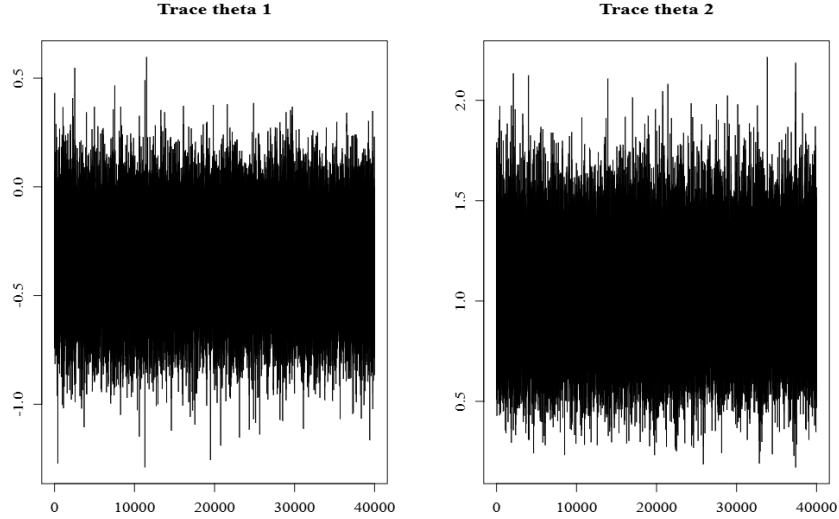


Figure 2: Trace plots for β_0 and β_1 showing good chain mixing.

- **Gelman–Rubin convergence diagnostic ($\hat{R}$):** The $\hat{R}$ statistic is 1.00 for both parameters, so this quantitatively proves that the variance between the chains is indistinguishable from the variance within the individual chains.
- **Autocorrelation (ACF):** The autocorrelation plots demonstrate rapid decay, dropping to zero and fluctuating around it within roughly 5 lags. This indicates that the sampler is generating independent draws rapidly, with no need for thinning.

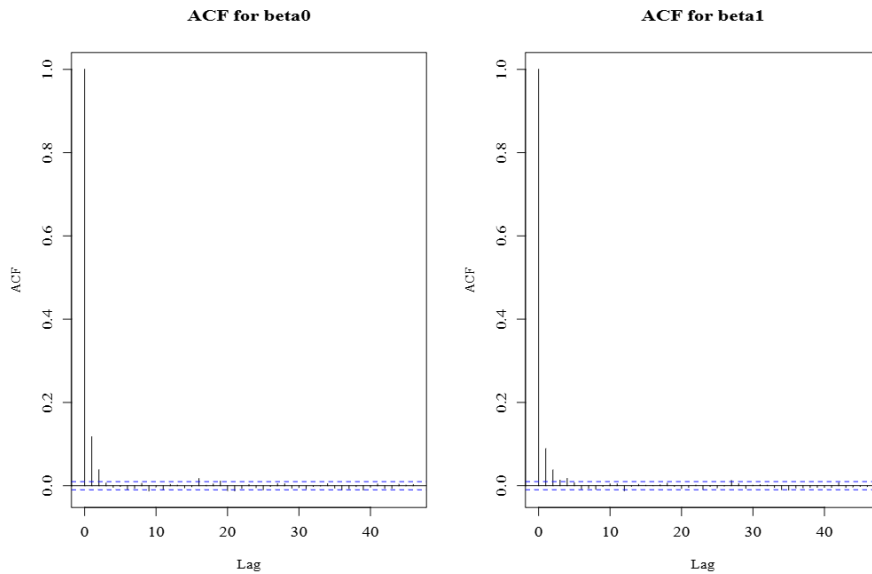


Figure 3: Autocorrelation functions for β_0 and β_1 showing rapid decay.

- **Effective Sample Size (ESS):** The computed effective sample sizes are exceptionally high (≈ 31877 for β_0 and ≈ 30269 for β_1). Out of the 40000 total post burn-in draws across all chains, roughly 75% are independent samples.

```

1 # 2. MCMC fitting and diagnostics
2 fit1 <- mod1$sample(data=data,
3   chains=4,
4   parallel_chains=4,
5   iter_warmup=1000,
6   iter_sampling=10000,
7   refresh = 0, # Silences the iteration progress updates
8 )
9 fit1$summary(c("beta0", "beta1"))
10 fit1$cmdstan_diagnose()
11
12 mcmc_full <- as_mcmc_list(fit1)
13 mcmc_chain <- mcmc_full[, c("beta0", "beta1")]
14 summary(mcmc_chain)
15
16 # trace plots
17 mcmc_mat <- as.matrix(mcmc_chain)
18 par(mfrow=c(1,2))
19 for (j in 1:ncol(mcmc_mat)) {
20   plot(mcmc_mat[,j], type="l", main=paste("Trace theta", j), ylab="", xlab="")
21 }
22
23 # ESS
24 ess <- effectiveSize(mcmc_mat)
25 print(ess)
26
27 # autocorrelation
28 par(mfrow=c(1, 2))
29 for (j in 1:ncol(mcmc_mat)) {
30   acf(mcmc_mat[,j], main=paste("ACF for", colnames(mcmc_mat)[j]))
31 }

```

Running MCMC with 4 parallel chains...

Chain 1 finished in 1.2 seconds.
 Chain 2 finished in 1.2 seconds.
 Chain 3 finished in 1.1 seconds.
 Chain 4 finished in 1.1 seconds.

All 4 chains finished successfully.
 Mean chain execution time: 1.1 seconds.
 Total execution time: 1.7 seconds.

```

> fit1$summary(c("beta0", "beta1"))
# A tibble: 2 × 10
  variable mean median sd mad q5 q95 rhat ess_bulk ess_tail
  <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 beta0 -0.340 -0.339 0.207 0.207 -0.680 -0.00277 1.00 30346. 26476.
2 beta1 1.04 1.03 0.249 0.247 0.639 1.46 1.00 30618. 24958.

```

```

> fit1$cmdstan_diagnose()

```

```
Checking sampler transitions treedepth.  
Treedepth satisfactory for all transitions.
```

```
Checking sampler transitions for divergences.  
No divergent transitions found.
```

```
Checking E-BFMI - sampler transitions HMC potential energy.  
E-BFMI satisfactory.
```

```
summary(mcmc_chain)  
Rank-normalized split effective sample size satisfactory for all parameters.
```

```
Rank-normalized split R-hat values satisfactory for all parameters.
```

```
Processing complete, no problems detected.
```

```
> summary(mcmc_chain)
```

```
Iterations = 1:10000  
Thinning interval = 1  
Number of chains = 4  
Sample size per chain = 10000
```

```
1. Empirical mean and standard deviation for each variable,  
   plus standard error of the mean:
```

	Mean	SD	Naive SE	Time-series SE
beta0	-0.3401	0.207	0.001035	0.001197
beta1	1.0362	0.249	0.001245	0.001430

```
2. Quantiles for each variable:
```

	2.5%	25%	50%	75%	97.5%
beta0	-0.7506	-0.4781	-0.3387	-0.1988	0.06022
beta1	0.5672	0.8640	1.0293	1.1979	1.54431

```
> print(ess)  
      beta0      beta1  
31877.37 30269.36
```

3 Posterior Inference

Posterior inference. Report posterior summaries for β_0 and β_1 : posterior mean, median, standard deviation, and 95% credible interval. Plot the marginal posterior distributions and the joint posterior samples in the (β_0, β_1) plane. Comment on the sign, magnitude, and uncertainty of the covariate effect, the posterior correlation between the two regression coefficients.

We extract the posterior summaries for the regression coefficients directly from our MCMC chains. The table below presents the posterior mean, median, standard deviation, and the 95% credible intervals (extracted from the 2.5% and 97.5% quantiles) for both β_0 and β_1 :

Parameter	Mean	Median	SD	2.5%	97.5%
β_0	-0.340	-0.339	0.207	-0.751	0.060
β_1	1.040	1.030	0.249	0.567	1.544

Table 1: Posterior summaries for the intercept (β_0) and slope (β_1).

Next, we visualize these posterior distributions. Figure 4 shows the marginal posterior histograms, while Figure 5 displays the joint posterior samples in the (β_0, β_1) plane using both a scatter plot and a 2D filled density plot.

Interpretation of the Covariate Effect:

- **Sign and Magnitude:** The posterior mean of β_1 is positive (≈ 1.04). This indicates a positive relationship between the covariate x and the probability of success ($y = 1$). In terms of odds, a one-unit increase in the covariate multiplies the odds of a positive outcome by a factor of $\exp(1.04) \approx 2.83$.
- **Uncertainty:** The standard deviation is relatively small (0.249), and the 95% credible interval $[0.567, 1.544]$ is strictly above zero. Therefore, we can confidently conclude that the covariate effect is positive and different from zero.

Posterior Correlation:

Observing the joint posterior plots, there is a visible negative correlation between β_0 and β_1 , which is mathematically confirmed by the correlation matrix of the posterior draws. Geometrically, the parameters become coupled as a steeper slope (β_1) necessitates a lower intercept (β_0) to fit the data points.

This posterior correlation could be mitigated by standardizing the covariate x_i , which would orthogonalize the intercept and slope. However, there is no need to standardize, as the HMC sampler exhibited excellent mixing and handled this correlation without any geometric pathology.

```

1 # 3. Posterior inference
2 fit1$summary(c("beta0", "beta1"))
3
4 # plot of the marginal posterior distributions
5 par(mfrow=c(1,2))
6 for (j in 1:ncol(mcmc_mat)) {
7   hist(mcmc_mat[,j], breaks=30,
8       main=paste("Posterior of", colnames(mcmc_mat)[j]),
9       xlab="", freq=FALSE)
10 }
11
12 # plots of the joint posterior distribution
13 par(mfrow=c(1,1))
14 plot(mcmc_mat[,1], mcmc_mat[,2],
15     main="Joint Posterior of beta0 and beta1",
16     xlab="beta0", ylab="beta1", pch=16, cex=0.5)
17 df <- as_draws_df(fit1$draws())
18 vars <- c("beta0", "beta1")
19 cor(as.matrix(df[, vars]))
20 ggplot(df, aes(x = beta0, y = beta1)) +
21   geom_density_2d_filled() +
22   theme_minimal()

```

```

> cor(as.matrix(df[, vars]))
      beta0      beta1
beta0 1.0000000 -0.2131551
beta1 -0.2131551 1.0000000

```

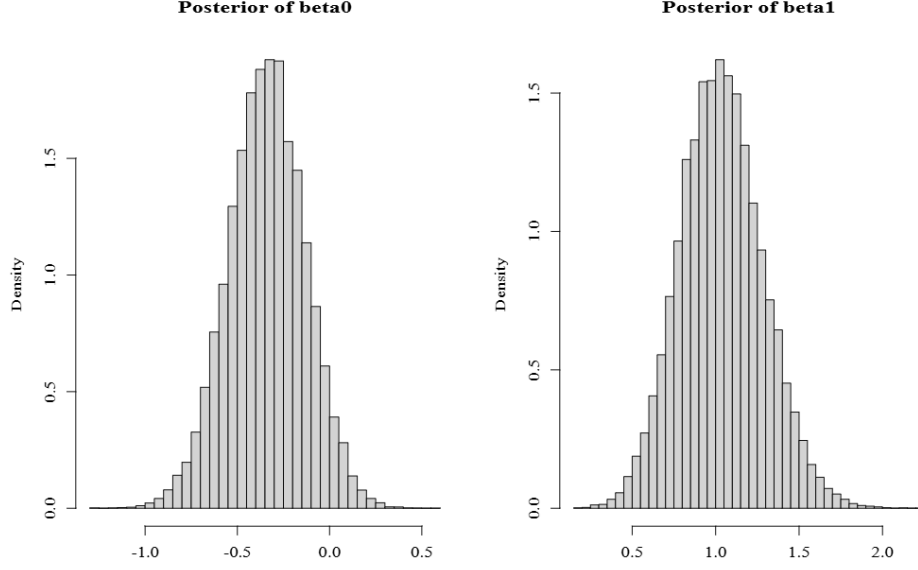


Figure 4: Marginal posterior distributions for β_0 and β_1 .

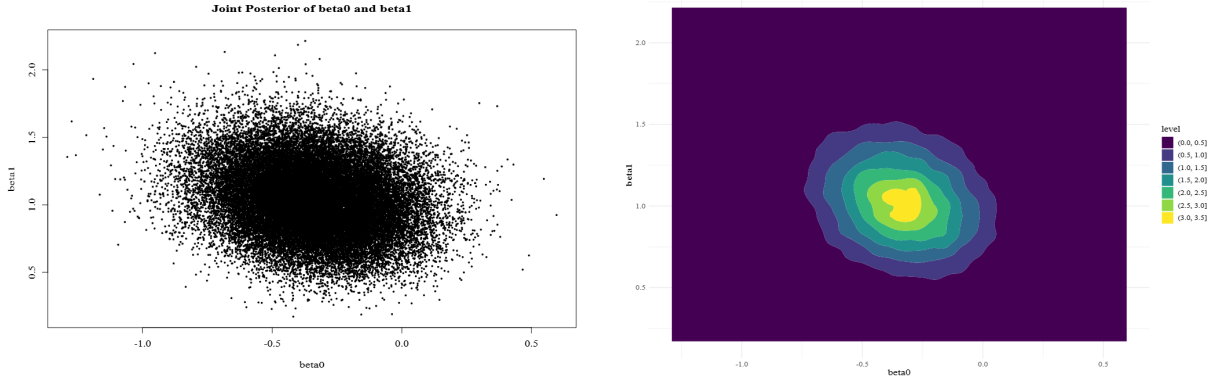


Figure 5: Joint posterior distribution of β_0 and β_1 represented as a scatter plot and a 2D density plot.

4 Posterior Predictive Inference

Posterior predictive inference. For a new covariate value x_{new} , use posterior simulation to approximate the posterior predictive distribution

$$p(y_{new}|x_{new}, D).$$

State the value of x_{new} you use. Report the posterior mean and 95% credible interval for

$$\pi_{new} = P(y_{new} = 1|x_{new}, \beta_0, \beta_1).$$

and report the posterior predictive probability

$$P(y_{new} = 1|x_{new}, D).$$

Produce a histogram or density plot of posterior draws of π_{new} .

We set the new covariate value at $x_{new} = 1.0$.

We compute the posterior predictive distribution by marginalizing over the posterior parameters:

$$p(y_{new}|x_{new}, D) = \int p(y_{new}|x_{new}, \beta_0, \beta_1)p(\beta_0, \beta_1|D) d\beta_0 d\beta_1$$

$$\approx \frac{1}{S} \sum_{s=1}^S (\pi_{new}^{(s)})^{y_{new}} (1 - \pi_{new}^{(s)})^{1-y_{new}} \quad (3)$$

where $\pi_{new}^{(s)} = \text{logit}^{-1}(\beta_0^{(s)} + \beta_1^{(s)}x_{new})$ is the success probability evaluated using the s -th MCMC sample. As we are interested in predicting a positive outcome, $P(y_{new} = 1|x_{new}, D)$, this expectation simplifies to:

$$P(y_{new} = 1|x_{new}, D) \approx \frac{1}{S} \sum_{s=1}^S \pi_{new}^{(s)} \quad (4)$$

Therefore, the posterior predictive probability is simply the sample mean of the distribution of π_{new} , resulting in 0.6643, with a 95% credible interval of [0.5358, 0.7822].

```

1 # 4. Posterior predictive inference
2 x_new <- 1.0
3 beta0_draws <- mcmc_mat[, "beta0"]
4 beta1_draws <- mcmc_mat[, "beta1"]
5
6 # plogis() is the inverse logit function
7 pi_new <- plogis(beta0_draws + beta1_draws * x_new)
8
9 # posterior mean and 95% credible interval for pi_new
10 pi_mean <- mean(pi_new)
11 pi_ci <- quantile(pi_new, probs = c(0.025, 0.975))
12
13 sprintf("Posterior mean of pi_new: %.4f", pi_mean)
14 sprintf("95%% Credible Interval for pi_new: [%.4f, %.4f]", pi_ci[1], pi_ci[2])
15
16 # plot the histogram and density
17 par(mfrow=c(1,1))
18 hist(pi_new, breaks=50, col="steelblue", border="white",
19      main=expression(paste("Posterior Distribution of ", pi[new])),
20      xlab=expression(pi[new]), freq=FALSE)
21 density_pi_new <- density(pi_new)
22 lines(density_pi_new, col="darkred", lwd=2)

```

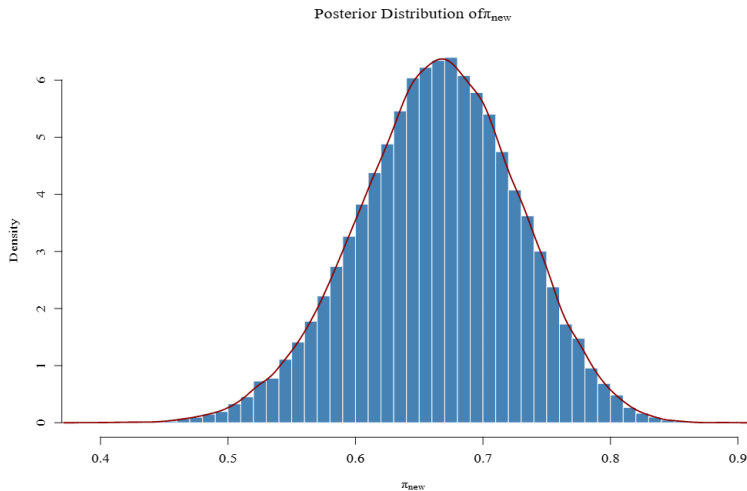


Figure 6: Posterior distribution of the success probability π_{new} at $x_{new} = 1.0$ with a KDE overlay.

```
> sprintf("Posterior mean of pi_new: %.4f", pi_mean)
[1] "Posterior mean of pi_new: 0.6643"

> sprintf("95%% Credible Interval for pi_new: [%.4f, %.4f]", pi_ci[1], pi_ci[2])
[1] "95% Credible Interval for pi_new: [0.5358, 0.7822]"
```

5 Model Comparison

Model comparison. Fit the intercept-only model

$$M_0 : \text{logit}(\pi_i) = \beta_0,$$

$$\beta_0 \sim \mathcal{N}(0, 3^2)$$

and compare it with the full model

$$M_1 : \text{logit}(\pi_i) = \beta_0 + \beta_1 x_i.$$

Use an appropriate Bayesian model-comparison criterion available from your chosen workflow. Interpret which model is better supported by the data and whether the covariate x_i improves predictive performance.

To fit the intercept-only model M_0 , we write another Stan model code and then compile it to sample 4 chains with 10000 points and a burn-in of 1000. The code structure is identical to M_1 , simply omitting the β_1 parameter and the covariate x :

```
1 data {
2   int<lower=1> N;
3   vector[N] x;
4   array[N] int<lower=0, upper=1> y;
5 }
6 parameters {
7   real beta0;
8 }
9 model {
10  beta0 ~ normal(0, 3);
11  y ~ bernoulli_logit(beta0);
12 }
13 generated quantities {
14  array[N] int y_rep;
15  vector[N] log_lik;
16  for (i in 1:N) {
17    y_rep[i] = bernoulli_logit_rng(beta0); // samples
18    log_lik[i] = bernoulli_logit_lpmf(y[i] | beta0); // log pmf
19  }
20 }
```

The sampling diagnosis performed by Stan returned a well-behaved HMC exploration with zero divergent transitions.

To perform the Bayesian model comparison, we evaluate the Expected Log Predictive Density (ELPD) using Leave-One-Out (LOO) validation, which is implemented in Stan library. LOO evaluates the predictive accuracy of the model by fitting it N separate times. In each iteration i , a single data point y_i is left out, the model is trained on the remaining $N - 1$ points (denoted as y_{-i}), and this newly trained model is used to predict the probability of the omitted point. The ELPD aggregates these results by summing the expected log-predictive probabilities for all N omitted points:

$$\text{ELPD} = \sum_{i=1}^N \log(p(y_i | y_{-i}))$$

Because actually refitting a Bayesian MCMC model N times is computationally prohibitive, the `loo` package in R approximates this exact process using Pareto Smoothed Importance Sampling (PSIS) applied directly to the point-wise log-likelihoods stored during our single model run.

We compute the LOO metrics for both models and evaluate their relative performance. The results show that the full model M_1 outperforms the intercept-only model M_0 . Specifically, `loo_compare` reports an ELPD difference of -9.9 for M_0 with a standard error of 4.2 . Because the magnitude of this difference is more than twice its standard error, we can conclude that including the covariate x_i improves the predictive accuracy, making M_1 a better model.

```

1 # 5. Model comparison
2 mod0 <- cmdstan_model("ex3/logistic_regressionM0.stan")
3
4 # we fit and sample from M0
5 fit0 <- mod0$sample(data=data,
6   chains=4,
7   parallel_chains=4,
8   iter_warmup=1000,
9   iter_sampling=10000,
10  refresh = 0,
11 )
12 fit0$summary(c("beta0"))
13 fit0$cmdstan_diagnose()
14
15 # comparison using LOO
16 log_lik_M1 <- fit1$draws("log_lik")
17 log_lik_M0 <- fit0$draws("log_lik")
18
19 # compute LOO for both models
20 loo_M1 <- loo(log_lik_M1)
21 loo_M0 <- loo(log_lik_M0)
22 comparison <- loo_compare(loo_M0, loo_M1)
23 print(comparison)

```

```

> fit0$summary(c("beta0"))
# A tibble: 1 × 10
  variable    mean median    sd    mad    q5    q95  rhat ess_bulk ess_tail
  <chr>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>    <dbl>    <dbl>
1 beta0    -0.204 -0.204 0.183 0.183 -0.504 0.0974 1.00    14506.    19211.

```

```

> print(comparison)
      elpd_diff se_diff
model2    0.0      0.0
model1   -9.9      4.2

```

6 Discussion

Discussion. Briefly summarize your conclusions. Your discussion should address whether the chains appear to have converged, whether the posterior uncertainty is substantial, whether the covariate effect is credibly different from zero, whether the posterior predictive probabilities are compatible with the observed data, and which model you prefer.

We can summarize our findings by addressing the core aspects of the analysis:

- **Chain Convergence:** The HMC chains exhibited convergence and good mixing. This is conclusively supported by stationary trace plots, $\hat{R}$ statistics of exactly 1.00, extremely high effective sample sizes, and a complete absence of divergent transitions.

- **Posterior Uncertainty and Covariate Effect:** The posterior uncertainty is quite low relative to the parameter magnitudes. The covariate effect β_1 is definitively positive and different from zero; its entire 95% credible interval $[0.567, 1.544]$ sits above zero, proving that x is a statistically significant predictor.
- **Predictive Compatibility:** The posterior predictive probabilities are highly compatible with the empirical data. Our simulation at $x_{new} = 1.0$ yielded a predicted success probability of roughly 66.4%. This aligns with our initial exploratory data analysis in Figure 1, which demonstrated that higher values of the covariate are strongly associated with the $y = 1$ outcome class.
- **Model Preference:** We prefer the full model M_1 over the intercept-only model M_0 . With an ELPD difference in the LOO evaluation of 9.9, M_1 is decisively better supported by the data than M_0 .